

• Introduction

• Data Structure

- A data structure is a specialised format for organizing, processing, retrieving and storing data.
- A logical or mathematical model to store and manipulate data efficiently.

• Common Operations on Data Structures

• Traversing -

- Traversing a DS means to visit the element stored in it.
- It visits the data in sequential and systematic manner.

• Searching -

- Searching means to find a particular element in the given data structure.
- It is considered successful when the required element is found.

• Insertion -

- It is the operation which we apply on all the data structures.
- Insertion means to add an element in the given data structure.

• Deletion

- It is the operation which we apply on all the data structures to delete an element in the given data.

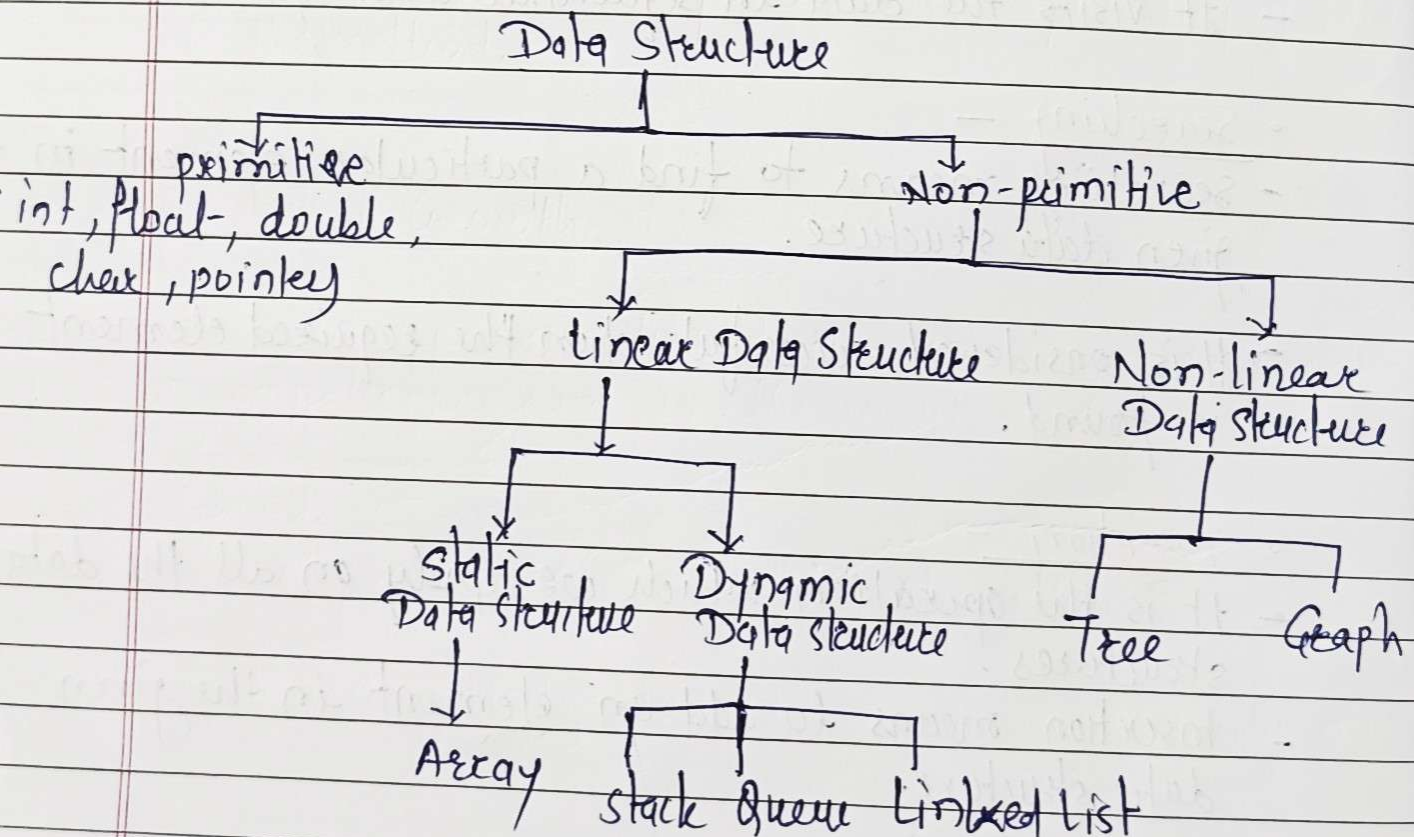
• Merging

- merging means combining to separate data into one.

• Sorting

- Arranging the data in a proper order either descending or ascending.
- Various sorting algorithms are there to sort the data.

• Types of - Data Structures



★ Linear Data structure

- Data elements are arranged sequentially, where each element is attached to its previous and next adjacent elements.

(1) Static Data structure

- This has a fixed memory size
- Easy to access.
- Ex. Array.

(2) Dynamic Data structure

- The size is not fixed.
- It can be randomly updated during the execution which may be considered efficient concerning the memory (space) complexity of the code.
- Ex. stack, Queue, linked list

★ Non-linear Data structure

- Data elements are not placed sequentially.
- In a non-linear data structure, we can't traverse all the elements in a single run only.
- Ex. Tree and Graphs.

• Need of Data Structure

1. Data structure modification is easy.
2. It requires less time.
3. Save storage memory space.
4. Data representation is easy.
5. Easy access to the large database.

• Complexities

- Complexities is a function that describes how long an algorithm takes in terms of the quantity of input it receives.

- Two complexities are there.

① Time Complexity

- A function that describes how long an algorithm takes in terms of the quantity of input it receives.

② Space Complexity

- A function that describes how much memory space an algorithm requires to the quantity of input to the method.

• Order of Growth

- It is an approximation of the time required to run a computer program as the input size increases.

- For ex.

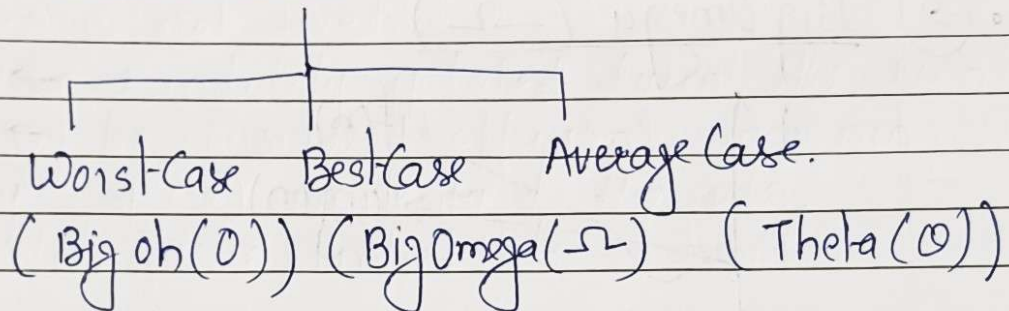
Algorithm A
↓
Time Complexity = $100n + 1$

Algorithm B
↓
Time Complexity = $n^2 + n + 1$

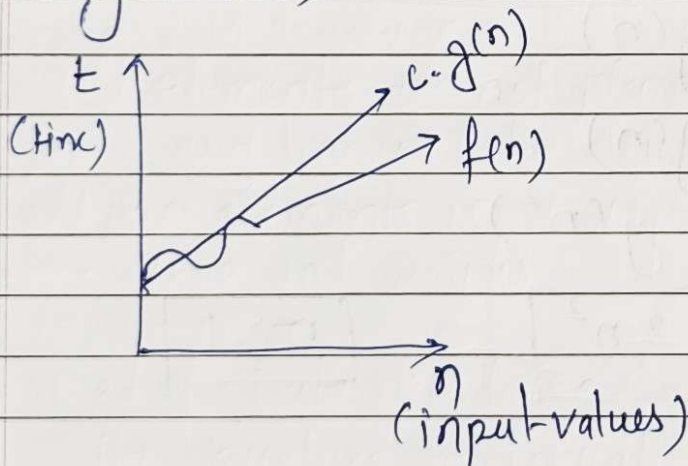
Input-Size	Runtime of A	Run time of B
10	1001	111
100	10001	10101
1000	10,0001	1001001
10000	1,000,001	$>10^{10}$
	$\boxed{n}$	$\boxed{n^2}$

- A order of growth is a set of functions whose asymptotic behaviour considered equivalent.

Asymptotic Notation



Big Oh (O)



→ Worst-Case
→ Upper Bound.
(Least Upper Bound)

$$f(n) = O(g(n))$$

$$f(n) \leq c \cdot g(n)$$

$$c > 0 \quad n \geq 0$$

$$f(n) = 2n^2 + n$$

$$f(n) = O(\quad)$$

$$2n^2 + n \leq c \cdot g(n)$$

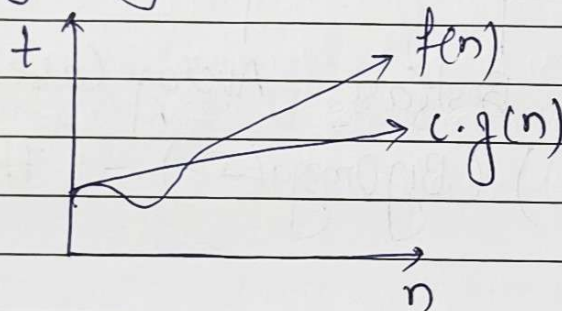
$$2n^2 + n \leq c \cdot g(n^2)$$

$$2n^2 + n \leq 2n^2$$

$$2n^2 + n \leq 3n^2$$

$$n \geq 1$$

• (2) Big Omega (Ω)



→ Best Case
→ Lower Bound
(Highest)

$$f(n) = \Omega g(n)$$

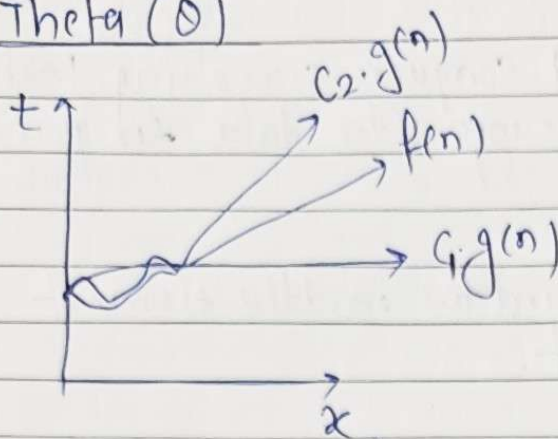
$$f(n) \geq c \cdot g(n)$$

$$2n^2 + n \geq c \cdot g(n^2)$$

$$2n^2 + n \geq 2n^2$$

$$n \geq 1$$

• (3) Theta (Θ)



→ Average case

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

$$2n^2 \leq 2n^2 + n \leq 3n^2$$

• Sorting & Searching

• Searching

→ Searching in data structures refers to the process of finding the required information from a collection of items.

→ Types - (1) Linear Search
(2) Binary Search.

• (1) Linear Search

→ Very basic and simple search algorithm

→ In linear search elements are searched by traversing on each element in data structure.

• ② Binary Search

- It is the divide and Conquer searching technique in which we have to arrange the data in particular format.
- After that we will find the middle element and compare with target element.
- If item not found then we will check the target element again.
- If it is greater than we search in right side of middle element.
- If it is less than we search in left side of middle element.

• Sorting

- Sorting is a method to arrange N elements in a particular format either in ascending or descending order.
- Types -
 - Bubble Sort
 - Selection Sort
 - Insertion Sort
 - Quick Sort
 - Merge Sort
 - Heap Sort
 - Radix Sort
 - Shell Sort

① Bubble Sort

- In Bubble sorting technique each pair of adjacent elements compared and elements are swapped if they are not in order.
- Best Case - $O(n)$
- Worst case time complexity = $O(n^2)$ - Auxiliary space - $O(1)$

Average Case

- Algorithm

- Traverse from left and compare adjacent elements and the higher one is placed at right side.

Pass L R
1 [5 | 2 | 3 | 4 | 6]

2 [2 | 5 | 3 | 4 | 6]

3 [2 | 3 | 5 | 4 | 6]

4 [2 | 3 | 4 | 5 | 6]

- In this way, the largest element is moved to the rightmost end at first.

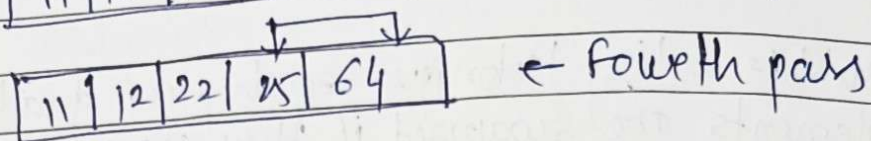
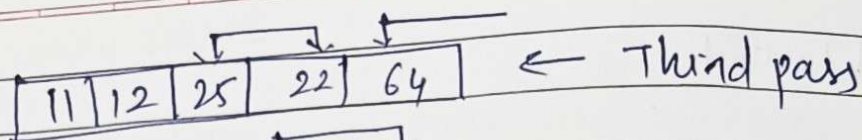
- This process is then continued to find the second largest and place it and so on until the data is sorted.

② Selection Sort

- Is a simple and efficient algorithm that works by repeatedly selecting the smallest element from the list and moving it to the sorted portion of the list.

→ Example - [64 | 25 | 12 | 22 | 11] - 1st pass

[11 | 25 | 12 | 22 | 64] - 1st pass



Time Complexity - $O(N^2)$ (Best, Worst, Average)
Auxiliary Space - $O(1)$

③ Insertion Sort

- Insertion Sort works similar to the sorting of playing cards in hands.
- The elements are splitted into a sorted and an unsorted part.
- Values from the sorted part are picked and placed at the correct position in the second part.

Step 1 - If the element is the first element assume that it is already sorted.

Step 2 - Pick the next element and store it separately in a key.

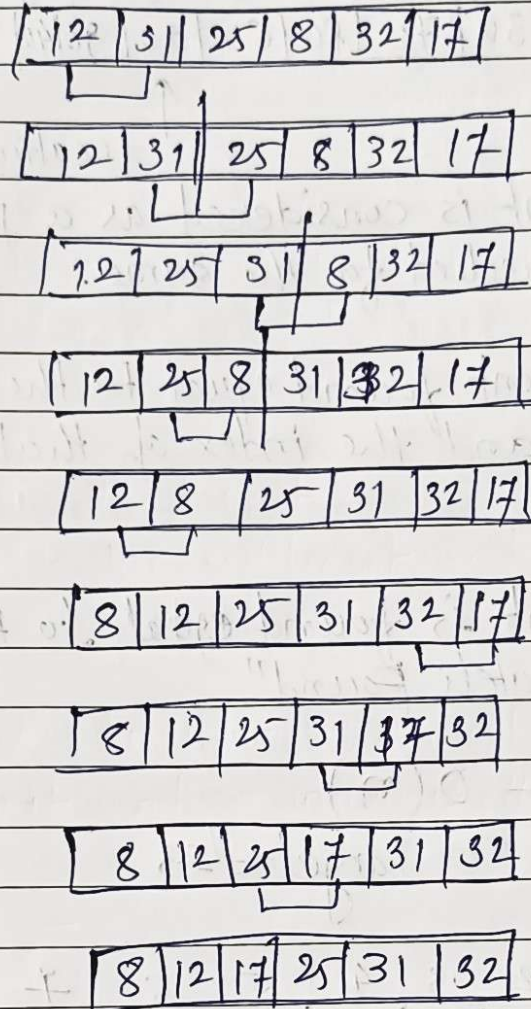
Step 3 - Now, compare the key with all elements in the sorted part.

Step 4 - If the element in the sorted array is smaller than the current element, then move to the next element. else, shift greater elements in the array towards the right.

Step 5 - Insert the value

Step 6 - Repeat until the array is sorted.

Example



⇒ Time Complexity

Best Case - $O(n)$

Average case $O(n^2)$

Worst Case $O(n^2)$

Space Complexity

⇒ $O(1)$

* Example:

① Linear Search -

0	1	2	3	4	5	6	7	8
10	50	30	70	80	60	20	90	40

20

location = 6

Find 20

- Every element is considered as a potential match for the key and checked for the same.
- If any element found equal to the key, the search is successful and the index of that element is returned.
- If no element is found equal to the key, the search yields "No match Found".

• Complexity - $O(n)$

② Binary Search = Target = 23

0	1	2	3	4	5	6	7	8	9
2	5	8	12	16	23	38	56	72	91

low = 0

mid = 4

High = 9

• key 23 > mid 16

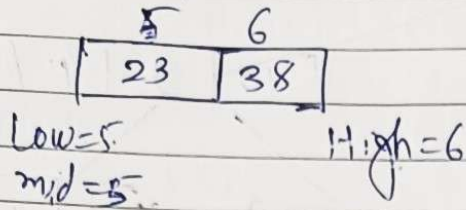
23	38	56	72	91
----	----	----	----	----

low = 5

mid = 7

High = 9

- key 23 < mid 56



$$\text{mid} = (\text{upper} + \text{lower}) / 2$$

Complexity - $O(\log n)$

- key matches with the mid.
Location = 5.

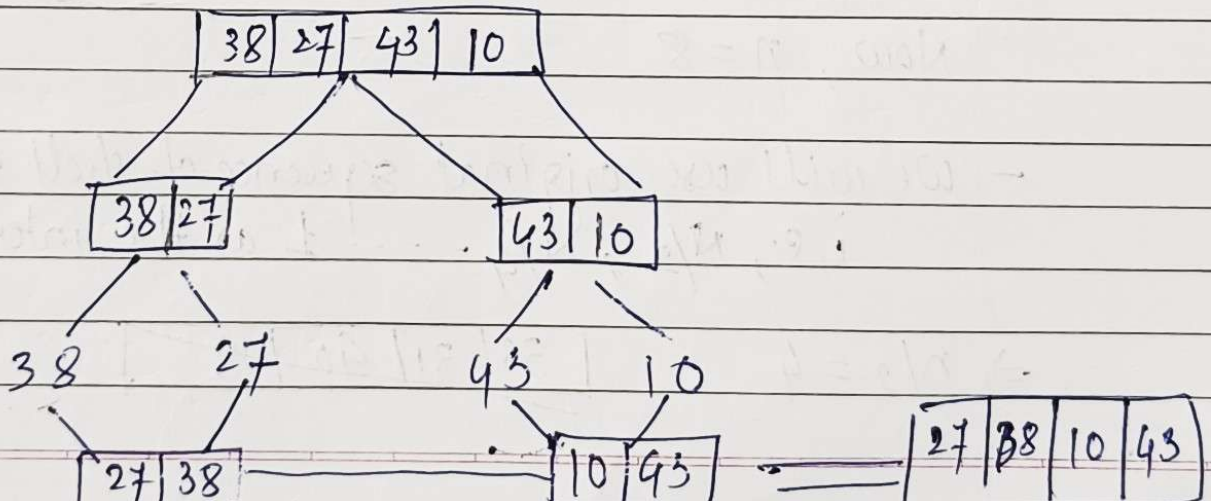
• Merging

- Data merging is the process of combining two or more similar records into single one.

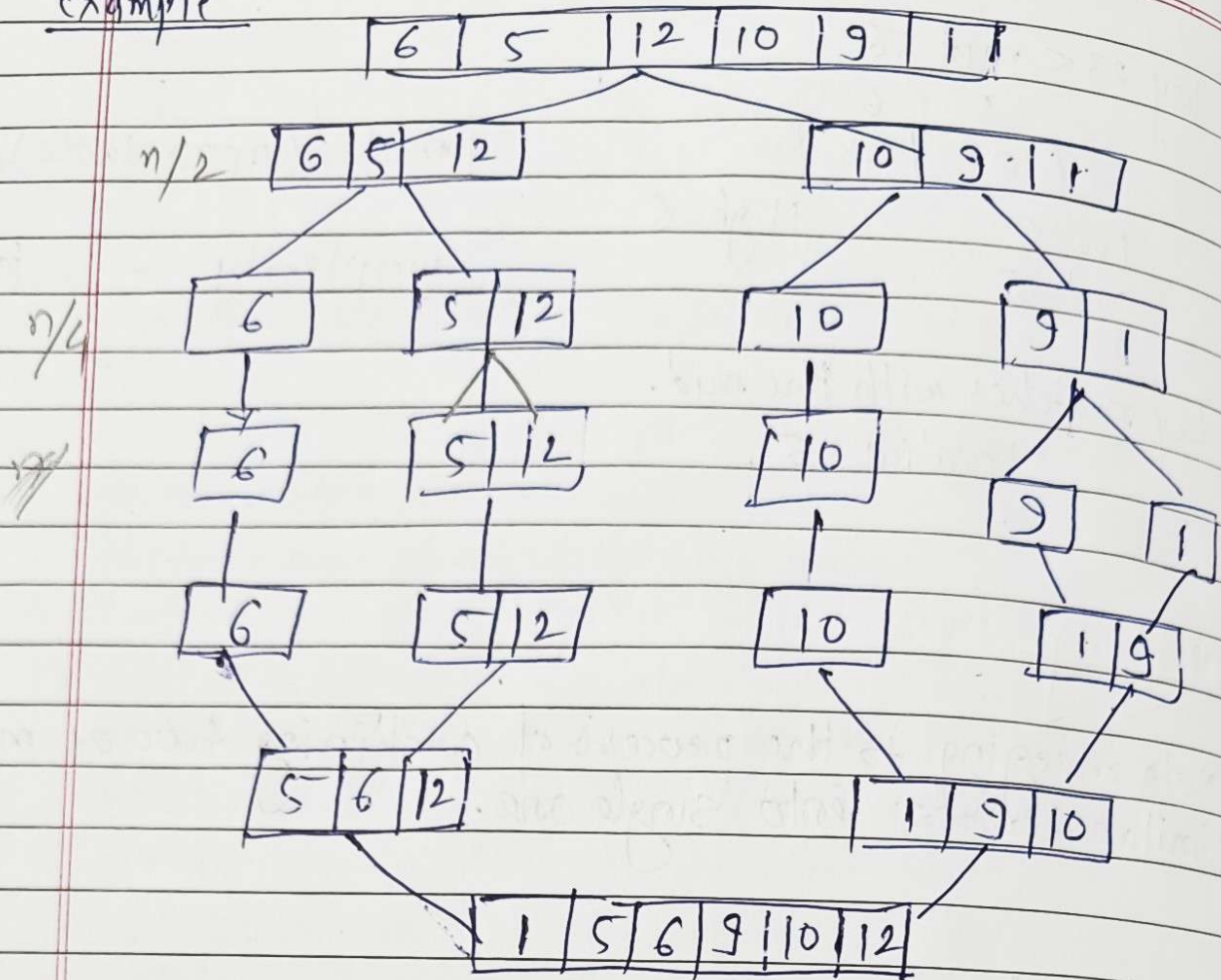
• ① merge Sort

- merge sort divides the elements into two parts and then sort each part and will merge sorted parts into one.

Example



Example



• (2) Shell Sort

- A variation of insertion sort.

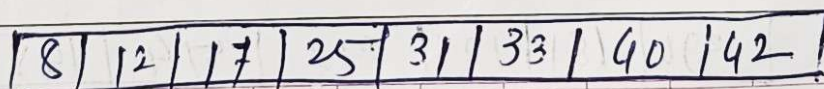
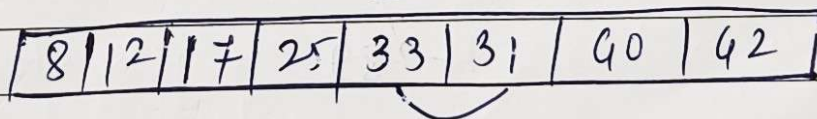
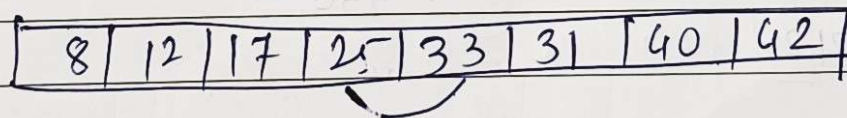
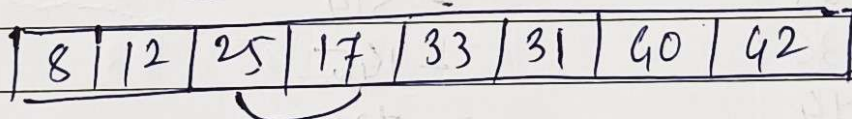
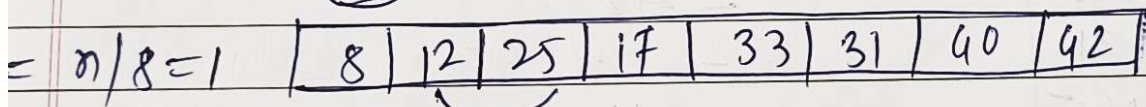
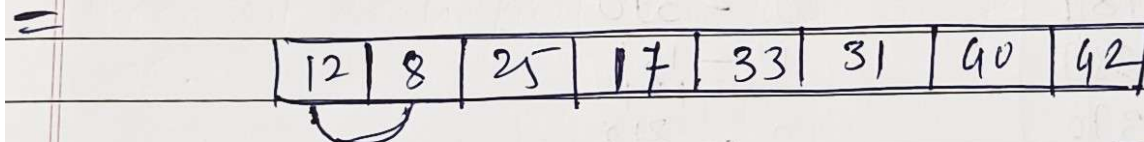
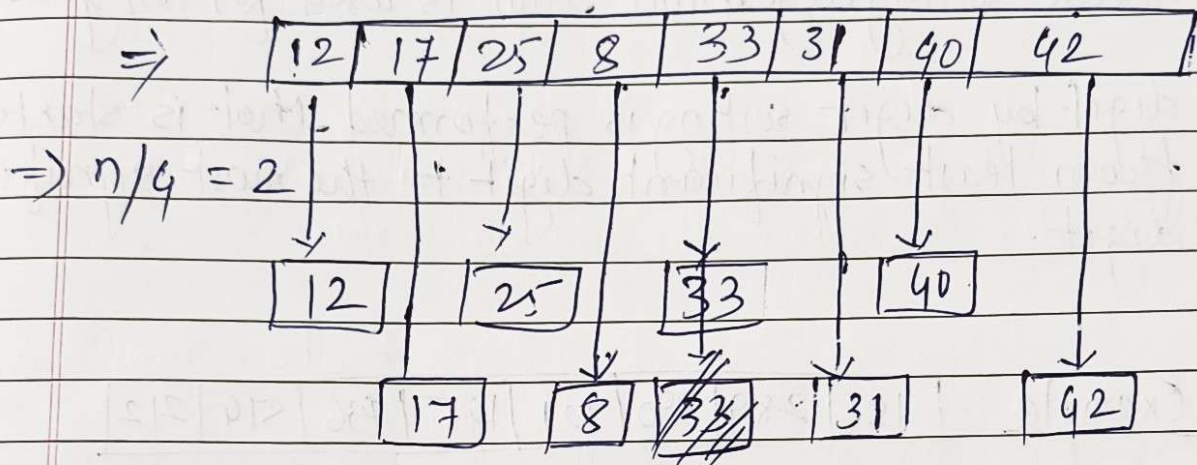
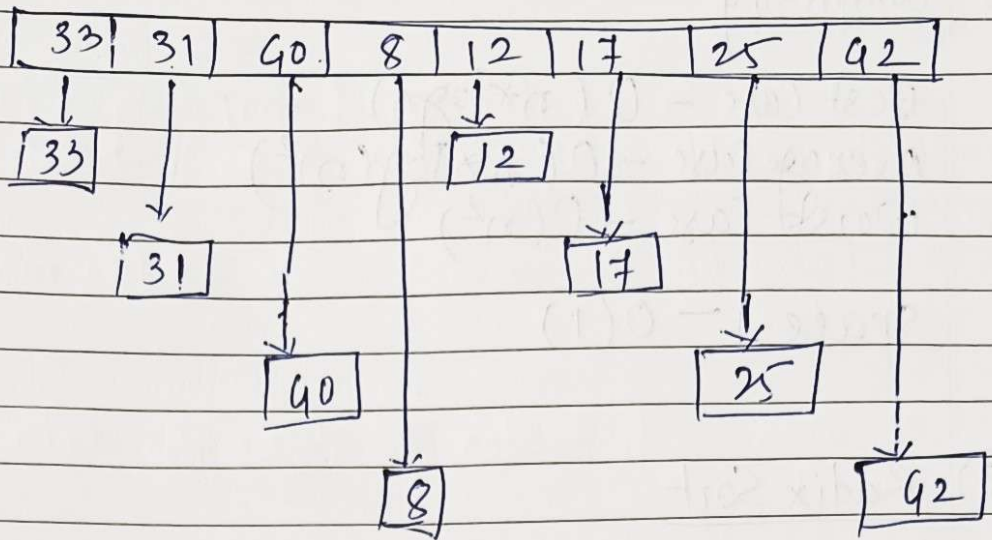
$[33, 31, 40, 8, 12, 17, 25, 42]$

Now, $n = 8$

- We will use original sequence of shell sort.
i.e., $n/2, n/4, \dots, 1$ as the intervals.

$\Rightarrow n/2 = 4$

~~$[33, 31, 40, 8]$~~



Complexity

Best Case - $O(n \log n)$
 Average Case - $O(n \log(n)^2)$
 Worst Case - $O(n^2)$
 Space - $O(1)$

• (3) Radix Sort

- Linear Sorting algorithm that is used for integers.
- digit-by-digit sorting is performed that is started from least significant digit to the most significant digit.

Example | 181 | 289 | 390 | 121 | 145 | 736 | 514 | 212 |

1st pass	181	0 - 390
	289	1 - 181
	390	2 - 212
	121	3 -
	145	4 - 514
	736	5 - 145
	514	6 - 736
	212	7 -
		8 -
		9 - 289

⇒ | 390 | 181 | 121 | 212 | 514 | 145 | 736 | 289 |

2nd pass

390
181
121
212
514
145
736
289

0 -
1 - 212
2 - 514
3 - 736
4 - 145
5 -
6 -
7 -
8 - 181
9 - 289
390

212	514	121	736	145	181	289	390
-----	-----	-----	-----	-----	-----	-----	-----

3rd pass

0 -
1 - 121
2 - 145
3 - 181
4 - 212, 289
5 - 390
6 -
7 - 514
8 -
9 - 736

121	145	181	212	289	390	514	736
-----	-----	-----	-----	-----	-----	-----	-----

Time Complexity

Best-Case

$\Omega(n+k)$

Average Case

$O(nk)$

Worst-Case

$O(nk)$

Space Complexity

$O(n+k)$

• Hashing

- Hashing is a technique of mapping a large chunk of data into small tables using a hashing function.
- Hashing function also known as the message digest function.
- It uses hash tables to store the data in an array format.
- Hashing is a two step process
 - ① The hash function converts the item into a small integer or hash value. This integer is used as an index to store the original data.
 - ② It stores the data in a hash table. You can use a hash key to locate data quickly.

$$\bullet \text{ hash} = \text{hashfunction}(\text{key})$$

$$\bullet \text{ index} = \text{hash} \% \text{array_size}$$

24, 52, 91, 67, 48, 83

$$\text{Hash function} = k \bmod 10$$

$$\Rightarrow 24 \bmod 10 = 4 \text{ — } 24 \text{ gets stored on 4th index of hash table.}$$

	0	$52 \bmod 10 = 2$
91	1	
52	2	$91 \bmod 10 = 1$
83	3	
24	4	$67 \bmod 10 = 7$
	5	
	6	$48 \bmod 10 = 8$
67	7	
48	8	$83 \bmod 10 = 3$
	9	

Hash Table

— Once the data elements are stored in hash table its easy to search them.

→ for ex. Search 67

$$\Rightarrow 67 \bmod 10 = 7$$

— So 67 should be search directly on the 7th index of hash table.